

Singleton Pattern — Thread-Safe Implementations

1. What is Singleton?

The **Singleton Design Pattern** ensures that a class has **only one instance** and provides a global access point to that instance.

Typical structure:

```
Client
  ↓
Singleton.getInstance()
  ↓
Same object every time
```

Common use cases can include:

- configuration
- shared application-level resources
- factories
- carefully designed caches

However, in Spring Boot, **Spring beans are singleton-scoped by default**, so you don't normally implement Singleton manually for ordinary Spring services.

2. Naive Singleton — Not Thread Safe

```
public class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {

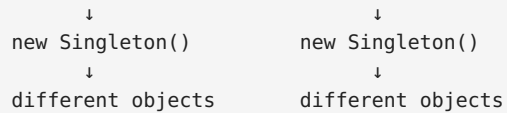
        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```

Looks correct in a single-threaded application.

But with multiple threads:

Thread A	Thread B
instance == null	instance == null



You can end up with **more than one instance**.

3. Synchronized Method

Simplest thread-safe implementation:

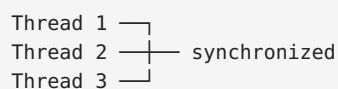
```
public class Singleton {  
  
    private static Singleton instance;  
  
    private Singleton() {  
    }  
  
    public static synchronized Singleton getInstance() {  
  
        if (instance == null) {  
            instance = new Singleton();  
        }  
  
        return instance;  
    }  
}
```

Advantage

Simple and thread-safe.

Disadvantage

Every call acquires the lock.



After the instance has already been created, synchronization is usually unnecessary overhead.

This leads to **Double-Checked Locking**.

4. Double-Checked Locking

The standard implementation is:

```
public class Singleton {  
  
    private static volatile Singleton instance;  
  
    private Singleton() {  
    }  
  
    public static Singleton getInstance() {
```

```

        if (instance == null) {

            synchronized (Singleton.class) {

                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }

        return instance;
    }
}

```

This is **thread-safe when implemented correctly**, including `volatile`.

5. Why Two Checks?

First check:

```
if (instance == null)
```

Avoids synchronization after the instance already exists.

Then:

```
synchronized (Singleton.class)
```

Only one thread can enter the critical section at a time.

Then the **second check**:

```
if (instance == null)
```

is necessary because another thread may have created the instance while the current thread was waiting for the lock.

Example

Thread A	Thread B
instance == null	
↓	
acquire lock	
↓	
create instance	
↓	
release lock	
	instance == null
	↓
	acquire lock
	↓
	second check
	↓
	instance != null

↓
don't create

Without the second check, Thread B could create another object after acquiring the lock.

6. Why `volatile` Is Necessary

This is the **important interview part**.

You might think:

```
private static Singleton instance;
```

is sufficient.

For correct double-checked locking, it isn't.

Object creation:

```
instance = new Singleton();
```

involves multiple conceptual steps:

1. Allocate memory
2. Initialize object
3. Assign reference

Without proper memory semantics, another thread could observe the reference before seeing a fully initialized object because of allowed reordering/visibility effects.

`volatile` provides the necessary **visibility and ordering guarantees** for the singleton reference.

So remember:

```
private static volatile Singleton instance;
```

Interview answer

"The `volatile` keyword is required for correct double-checked locking because it prevents problematic instruction reordering and ensures visibility of the initialized reference across threads."

7. Bill Pugh Singleton

This is a cleaner approach using a **static nested class**.

```
public class Singleton {  
    private Singleton() {  
    }  
    private static class Holder {
```

```
        private static final Singleton INSTANCE =
            new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

Why is this useful?

The JVM initializes the nested class lazily when it is first actively used.

So:

```
Application starts
↓
Singleton class loaded
↓
Holder not initialized
↓
getInstance()
↓
Holder initialized
↓
INSTANCE created
```

Class initialization is guaranteed to be thread-safe by the JVM.

Therefore you don't need:

`synchronized`

or:

`volatile`

in this implementation.

8. Why is Bill Pugh Thread Safe?

Because JVM **class initialization is synchronized by the JVM specification**.

Multiple threads can call:

```
Singleton.getInstance();
```

but initialization of:

```
Holder.INSTANCE
```

happens safely once.

This is one of the main reasons the Bill Pugh approach is preferred over manually synchronized implementations when you actually need a classic Singleton.

9. Enum Singleton

The simplest robust Singleton implementation in Java is often an enum:

```
public enum Singleton {  
  
    INSTANCE;  
  
    public void process() {  
        System.out.println("Processing");  
    }  
}
```

Usage:

```
Singleton.INSTANCE.process();
```

There is only one enum constant:

```
INSTANCE
```

The JVM handles enum instance creation and initialization.

10. Why Enum Singleton is Strong

Enum Singleton provides strong guarantees around:

Thread safety

Enum initialization is handled safely by the JVM.

Serialization

Enums have special serialization semantics, so the normal Singleton serialization problem is avoided.

Reflection

Enums also have special restrictions around reflective construction, making them more resistant to common reflection-based Singleton-breaking techniques.

Therefore:

Enum Singleton is often the simplest and most robust implementation when you control the class design and enum semantics are acceptable.

11. Comparison

Implementation	Thread Safe	Lazy	Complexity
Naive	✗	✓	Simple

Synchronized method	✓	✓	Simple
Double-checked locking	✓	✓	Medium
Bill Pugh	✓	✓	Simple
Enum	✓	Initialization semantics differ from classic lazy singleton	Simplest

For interviews, the key distinction is:

Double-Checked Locking

```
volatile + synchronized + double check
```

Bill Pugh

```
static nested class + JVM class initialization
```

Enum

```
enum INSTANCE
```

12. Singleton and Spring Boot

This is **very important for your interview**.

Spring beans are singleton-scoped by default.

For example:

```
@Service
public class PaymentService {
}
```

By default, Spring creates one bean instance per Spring `ApplicationContext`.

Conceptually:

```
Spring Container
  ↓
PaymentService Bean
  ↓
same bean returned to consumers
```

So you normally **do not write**:

```
PaymentService.getInstance();
```

Spring handles the lifecycle and dependency injection.

13. Important Difference

Don't say:

“Spring Singleton means there is one object in the entire JVM.”

That is too broad.

Spring's singleton scope means:

One bean instance per Spring IoC container/application context.

If there are multiple application instances:

```
Server 1 → Spring Context → PaymentService instance
Server 2 → Spring Context → PaymentService instance
Server 3 → Spring Context → PaymentService instance
```

There are multiple physical objects.

This is especially important when discussing **distributed systems**.

14. Singleton Does NOT Mean Thread Safe

Very important.

Suppose Spring has one instance:

```
@Service
public class PaymentService {

    private int count = 0;
}
```

Multiple request threads can access the same bean concurrently.

```
Thread A ┌──┐
Thread B ┌──┐ PaymentService singleton
Thread C ┌──┐
```

If `count` is mutable and updated unsafely, you can get race conditions.

So:

Singleton scope and thread safety are separate concerns.

For stateless Spring services, a good practice is to avoid mutable shared instance state.

15. Singleton and Your BPS Project

Imagine:

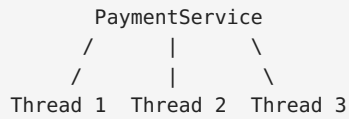
```
@Service
public class PaymentService {

    public PaymentResponse process(PaymentRequest request) {
        ...
    }
}
```

```
}
```

Spring creates one `PaymentService` bean per application context.

Requests can then arrive concurrently:



Therefore the service should generally be **stateless**.

Avoid:

```
@Service
public class PaymentService {

    private PaymentRequest currentRequest; // dangerous
}
```

because one request thread could overwrite state used by another.

Prefer:

```
@Service
public class PaymentService {

    public PaymentResponse process(
        PaymentRequest request) {
        // local variables
    }
}
```

Local variables belong to each thread's execution context.

16. Common Interview Questions

Q1. What is Singleton?

A creational design pattern that ensures a class has one instance and provides a global access point to it.

Q2. Is this implementation thread safe?

```
if (instance == null) {
    instance = new Singleton();
}
```

No.

Q3. How do you make it thread safe?

Possible approaches:

```
synchronized method
Double-checked locking
```

Bill Pugh holder
Enum

Q4. Why use `volatile` in double-checked locking?

To guarantee visibility and prevent problematic reordering around publication of the singleton instance.

Q5. Why do we check `instance == null` twice?

The first check avoids locking when the instance already exists. The second check prevents multiple threads that were waiting for the lock from creating multiple instances.

Q6. Why is Bill Pugh thread safe?

The JVM guarantees thread-safe class initialization, and the singleton instance is initialized when the holder class is initialized.

Q7. Why is enum Singleton considered robust?

JVM-managed enum initialization gives thread safety, and enum serialization/reflection behavior avoids several common ways of breaking a traditional Singleton.

Q8. Is a Spring singleton thread-safe?

No, not automatically.

Singleton scope means one instance per Spring container; thread safety depends on how that bean handles shared state.

17. Singleton Serialization Problem

Suppose you use:

```
class Singleton implements Serializable {  
    ...  
}
```

Serialization/deserialization can create another instance unless you handle it appropriately.

Traditional Singleton may need:

```
private Object readResolve() {  
    return INSTANCE;  
}
```

Enum Singleton avoids this problem through Java's special enum serialization mechanism.

This is a good advanced follow-up.

18. Singleton and Reflection

Traditional private constructors aren't an absolute guarantee against all reflective attacks.

Conceptually:

```
Constructor<Singleton> constructor =  
    Singleton.class.getDeclaredConstructor();  
  
constructor.setAccessible(true);
```

Reflection can potentially invoke the private constructor.

Enum types receive special protection against reflective construction.

This is another reason enum Singleton is considered robust.

19. Best Practice

For modern Java:

When you genuinely need a classic Singleton

Enum is often the simplest robust choice.

```
public enum ConfigManager {  
    INSTANCE;  
}
```

If you need lazy initialization while retaining a normal class

Bill Pugh Holder is a clean option.

```
private static class Holder {  
    static final Singleton INSTANCE = new Singleton();  
}
```

If interviewer specifically asks for DCL

Know this exact structure:

```
private static volatile Singleton instance;  
  
public static Singleton getInstance() {  
    if (instance == null) {  
        synchronized (Singleton.class) {  
            if (instance == null) {  
                instance = new Singleton();  
            }  
        }  
    }  
    return instance;  
}
```

20. One More Important Design Question

Why is Singleton sometimes considered problematic?

Because it introduces **global shared state** and can make:

- testing harder
- dependencies less explicit
- concurrency harder
- lifecycle management less clear

Spring's dependency injection generally gives you the same practical "one instance per context" behavior while keeping dependencies explicit:

```
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }
}
```

This is usually preferable to manually calling:

```
PaymentRepository.getInstance();
```

Interview-ready answer

"Singleton ensures that only one instance of a class is created and provides a global access point. A naive implementation isn't thread-safe because two threads can observe a null instance simultaneously. Double-checked locking solves this by checking outside and inside a synchronized block, and the instance must be declared `volatile` for correct publication. The Bill Pugh approach uses a static nested holder class and relies on JVM class initialization for thread safety and lazy initialization. An enum Singleton is even simpler and provides strong JVM guarantees around initialization and serialization. In Spring Boot, I normally don't implement Singleton manually because Spring beans are singleton-scoped by default, although that means one instance per Spring application context, not globally across multiple application instances."

For your interview, memorize these three patterns:

```
Double-Checked Locking
→ volatile + synchronized + 2 checks

Bill Pugh
→ static nested Holder class

Enum
→ INSTANCE
```

And remember the **most likely cross-question**:

“If Spring beans are Singleton, does that mean your `@Service` is thread-safe?”

Answer:

No. One shared bean instance can be accessed by many request threads, so the bean should generally be stateless or explicitly synchronize/protect any shared mutable state.